

✓ The Price is Right

Week 7 Day 2

Selecting our model and evaluating the base model against the task.

Keep in mind: our base model has 8 billion params, quantized down to 4 bits Compared with GPT-5 at TRILLIONS of params!

```
1 !pip install -q --upgrade bitsandbytes
```

43.1/43.1 MB 17.1 MB/s eta 0:00:00

```
1 !wget -q https://raw.githubusercontent.com/ed-donner/llm_engineering/main/week7/uti
```

```
1 # imports
2
3 import os
4 import re
5 import math
6 from tqdm import tqdm
7 from google.colab import userdata
8 from huggingface_hub import login
9 import torch
10 import transformers
11 from transformers import AutoModelForCausalLM, AutoTokenizer, BitsAndBytesConfig, T
12 from peft import LoraConfig, PeftModel
13 from datasets import load_dataset, Dataset, DatasetDict
14 from datetime import datetime
15 import matplotlib.pyplot as plt
16 from util import evaluate
```

```
1 # Constants
2
3 BASE_MODEL = "google/gemma-7b"
4 #BASE_MODEL = "meta-llama/Llama-3.2-3B"
5 PROJECT_NAME = "price"
6
7 LITE_MODE = True
8
9 DATA_USER = "ed-donner"
10 DATASET_NAME = f"{DATA_USER}/items_prompts_lite" if LITE_MODE else f"{DATA_USER}/it
11
12 # Hyper-parameters
13
14 QUANT_4_BIT = True
```

✓ Log in to HuggingFace

If you don't already have a HuggingFace account, visit <https://huggingface.co> to sign up and create a token.

Then select the Secrets for this Notebook by clicking on the key icon in the left, and add a new secret called `HF_TOKEN` with the value as your token.

```
1 # Log in to HuggingFace
2
3 hf_token = userdata.get('HF_TOKEN')
4 login(hf_token, add_to_git_credential=True)
```

✓ Load our data

We uploaded it to Hugging Face, so it's easy to retrieve it now

```
1 dataset = load_dataset(DATASET_NAME)
2 train = dataset['train']
3 val = dataset['val']
4 test = dataset['test']
```

```
1 train[0]
```

```
{'prompt': 'What does this cost to the nearest dollar?\n\nTitle: Schlage F59 & 613 Andover Interior Knob (Deadbolt Included) \nCategory: Home Hardware \nBrand: Schlage \nDescription: A single-piece oil-rubbed bronze knob that mounts to a deadbolt for secure, easy interior door use. \nDetails: Designed for a 4" minimum center-to-center door prep, it offers a lifetime mechanical and finish warranty and comes ready for quick installation.\n\nPrice is $',
 'completion': '64.00'}
```

✓ Prepare our Base Llama Model for evaluation

Load our base model with 4 bit quantization and try out 1 example

```
1 ## pick the right quantization
2
3 if QUANT_4_BIT:
4     quant_config = BitsAndBytesConfig(
5         load_in_4bit=True,
6         bnb_4bit_use_double_quant=True,
7         bnb_4bit_compute_dtype=torch.bfloat16,
8         bnb_4bit_quant_type="nf4"
9     )
10 else:
11     quant_config = BitsAndBytesConfig(
12         load_in_8bit=True,
13         bnb_8bit_compute_dtype=torch.bfloat16
14     )
```

```
1 # Load the Tokenizer and the Model
2
3 tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL, trust_remote_code=True)
4 tokenizer.pad_token = tokenizer.eos_token
5 tokenizer.padding_side = "right"
6
7 base_model = AutoModelForCausalLM.from_pretrained(
8     BASE_MODEL,
9     quantization_config=quant_config,
10    device_map="auto",
11 )
12 base_model.generation_config.pad_token_id = tokenizer.pad_token_id
13
14 print(f"Memory footprint: {base_model.get_memory_footprint() / 1e9:.1f} GB")
```

```

config.json: 100%                                629/629 [00:00<00:00, 62.9kB/s]
tokenizer_config.json: 100%                      33.6k/33.6k [00:00<00:00, 2.79MB/s]
tokenizer.json: reconstructing file: 100%         17.5MB / 17.5MB, 1.73MB/s
tokenizer.json: downloading bytes:               7.44MB, 732kB/s
special_tokens_map.json: 100%                   636/636 [00:00<00:00, 65.1kB/s]
model.safetensors.index.json: 100%              20.9k/20.9k [00:00<00:00, 2.08MB/s]
Download complete: :      14.8GB, 4.43MB/s
Reconstruction complete: 100%    17.1GB / 17.1GB, 24.8MB/s
Fetching 4 files: 100%                        4/4 [18:37<00:00, 183.56s/it]
Loading weights: 100%                        254/254 [01:09<00:00, 5.56it/s]
generation_config.json: 100%                  137/137 [00:00<00:00, 12.1kB/s]
Memory footprint: 5.4 GB

```

```

1 def model_predict(item):
2     inputs = tokenizer(item["prompt"], return_tensors="pt").to("cuda")
3     with torch.no_grad():
4         output_ids = base_model.generate(**inputs, max_new_tokens=8)
5         prompt_len = inputs["input_ids"].shape[1]
6         generated_ids = output_ids[0, prompt_len:]
7         return tokenizer.decode(generated_ids)

```

```
1 test[0]
```

```

{'prompt': 'What does this cost to the nearest dollar?\n\nTitle: Excess V2 Distortion/Modulation Pedal \nCategory: Music Pedals \nBrand: Old Blood Noise \nDescription: A versatile pedal offering distortion and three modulation modes -delay, chorus, and harmonized fifths-with full control over signal routing and expression. \nDetails: Features include separate gain, tone, and volume controls; time, depth, and volume per modulation; order switching, soft-touch bypass, and expression jack for dynamic control.\n\nPrice is $',
'completion': '219.0'}

```

```
1
```

```
1 model_predict(test[0])
```

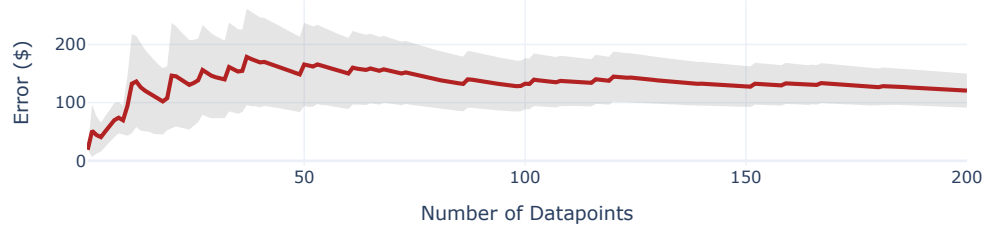
```
'199.99\n\nI'
```

Evaluation!

Trying out our base Llama 3.2 model against the Test dataset

```
1 evaluate(model_predict, test)
```

Model Predict Error: 120.66 ± 29.29



Model Predict results

Error: \$120.66 **MSE:** 59,221 **r²:** -169.5%

